

AI Identity Document Screening System

Complete Technical Architecture, Code Breakdown & Documentation

Repository: <https://github.com/vamshi242005/AI-Based-Fake-Identity-Document-Screening-System>
Supported Documents: Aadhaar Card | PAN Card | Passport | Voter ID Card

1. Executive Summary & Overview

The **AI-Based Fake Identity Document Screening System** is a full-stack web application designed to automatically inspect Indian identity documents and determine their authenticity using client-side Computer Vision models and server-side Large Language Model (LLM) reasoning.

When an uploaded document is flagged as **FAKE**, an integrated AI assistant powered by Google Gemini provides real-time explanations detailing common visual, structural, and cryptographic red flags specific to that document type.

2. Key Features & Supported Documents

Document Type	Model Path	Supported Classes	Security Red Flags Checked
Aadhaar Card	/model-aadhaar	ORIGINAL / FAKE	UIDAI Emblem, 12-digit format, QR code clarity, micro-text borders.
PAN Card	/model-pancard	ORIGINAL / FAKE	Income Tax Department seal, 10-character alphanumeric PAN, hologram placement.
Passport	/model-passport	ORIGINAL / FAKE	MRZ (Machine Readable Zone) lines, Republic of India emblem, ghost image.
Voter ID Card	/model-voterid	ORIGINAL / FAKE	EPIC number format, Election Commission logo, photo lamination integrity.

3. System Architecture & Workflow

System Execution Flow

1. **Authentication:** User registers/logs in via `login.html` (session persisted in `sessionStorage`).
2. **Model Preloading:** `script.js` preloads all 4 Teachable Machine TensorFlow.js models into browser GPU/memory upon startup.
3. **Classification:** User selects document type and uploads an image. `predictImage()` runs instant WebGL inference.
4. **Result Rendering:** Displays **ORIGINAL** or **FAKE** status with exact confidence percentage.
5. **AI Explanation Proxy:** If flagged as FAKE, frontend sends a POST request to `http://localhost:5001/api/explain-fake` .
6. **Gemini Explanation:** Express backend securely queries Google Gemini using persona system instructions and returns structured red-flag explanations.

4. Detailed Code Breakdown

A. Authentication Controller (`login.html` & `login.js`)

Handles local user registration and secure session protection before loading the main screener dashboard.

```
// login.js - Key Session Check & Authentication
function checkAuth() {
  const isLoggedIn = sessionStorage.getItem('isLoggedIn');
  if (!isLoggedIn && window.location.pathname.includes('index.html')) {
    window.location.href = 'login.html';
  }
}

function loginUser(username, password) {
  const usersDB = JSON.parse(localStorage.getItem('usersDB') || '{}');
  if (usersDB[username] && usersDB[username].password === hashPassword(password)) {
    sessionStorage.setItem('isLoggedIn', 'true');
    window.location.href = 'index.html';
  } else {
    showError("Invalid username or password");
  }
}
```

B. Frontend Classification & Multi-Model Engine (`script.js`)

Manages preloading of TensorFlow.js models, image rendering on canvas, and dataset label normalization.

```
// script.js - Centralized Document Registry & Concurrent Preloading
const DOC_TYPES = {
  aadhaar: { name: "Aadhaar Card", modelPath: "./model-aadhaar/" },
  pancard: { name: "PAN Card", modelPath: "./model-pancard/" },
  passport: { name: "Passport", modelPath: "./model-passport/" },
  voterid: { name: "Voter ID Card", modelPath: "./model-voterid/" }
};

let loadedModels = {};

async function preloadAllModels() {
  const loadPromises = Object.keys(DOC_TYPES).map(async (docKey) => {
    const modelURL = DOC_TYPES[docKey].modelPath + "model.json";
    const metadataURL = DOC_TYPES[docKey].modelPath + "metadata.json";
    loadedModels[docKey] = await tmImage.load(modelURL, metadataURL);
  });
  await Promise.all(loadPromises);
}

// Normalizes dataset class names (e.g. "ORIGINAL VOTEID" -> "ORIGINAL")
function normalizeClassName(className) {
  const upper = className.toUpperCase();
  if (upper.includes("ORIGINAL")) return "ORIGINAL";
  if (upper.includes("FAKE")) return "FAKE";
  return className;
}
```

C. Express Backend Proxy & Gemini AI (`server/server.js`)

Maintains server-side security by wrapping Gemini API requests with system instructions and fallback handlers.

```
// server/server.js - System Instructions & Gemini API Proxy
const SYSTEM_INSTRUCTION = `You are a document verification assistant. You explain,
in simple and professional language, common visual and structural signs that make
identity documents (Aadhaar, PAN, Passport, Voter ID) look fake based strictly on
the document type and classification confidence provided. Stay strictly on-topic.`;

app.post('/api/explain-fake', async (req, res) => {
  try {
    const { docType, confidence } = req.body;
    const prompt = `A ${docType} was classified as FAKE with ${confidence * 100}% confi
    const explanation = await callGemini(prompt);
    res.json({ success: true, explanation });
  } catch (error) {
    // Canned fallback output in case of rate-limiting or network issues
    res.json({ success: true, explanation: CANNED_FALLBACK });
  }
});
```

5. How to Run the Project

Execution Steps

1. **Start Frontend HTTP Server (Port 5000):**

```
python -m http.server 5000
```

 inside root directory.

2. **Start Backend Express Server (Port 5001):**

```
cd server && npm start
```

3. **Access in Browser:**

Open <http://localhost:5000/login.html>